GitHub Copilot for Testing and Refactoring

Use AI to generate tests, uncover edge cases, and refactor code — while you stay the final reviewer.

TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 11 OVERVIEW

Testing consumes significant developer time — AI can accelerate test creation, refactoring, and code review.

- Building on Module 10's fundamentals, you'll now use Copilot for testing-specific and refactoring-specific workflows — generating tests, finding edge cases, and cleaning up code.

DURATION & LAB

1 Hour Theory

AI-assisted testing, refactoring, code review.

45 Min Lab

AI-Assisted Test Generation.

Scenario

Generate unit tests, edge cases, and mocks for a service.

AI GENERATES

Unit tests, edge cases, mocks, and refactoring suggestions

YOU REVIEW

Correctness, security, quality, and maintainability

THE PAYOFF

Higher coverage, cleaner code, faster delivery

Enterprise scenario: in a large banking application, Copilot helps teams generate high-quality tests, cover edge cases, and refactor legacy code faster.

KEY TAKEAWAY AI is your assistant, not a replacement. Your judgment ensures enterprise-quality tests and code.

WHY TESTING AUTOMATION MATTERS

Automated testing is essential for delivering high-quality software faster, with confidence and efficiency.

WITHOUT TEST AUTOMATION

- Slow and manual — tests delay feedback
- More bugs escape to production
- High maintenance cost over time
- Risky, low-confidence releases

VS

WITH TEST AUTOMATION

- Fast, early feedback on every change
- Better coverage across more scenarios
- Cost-effective — enables CI/CD
- Confident, reliable releases

THE BUSINESS & TECH IMPACT

Faster Feedback

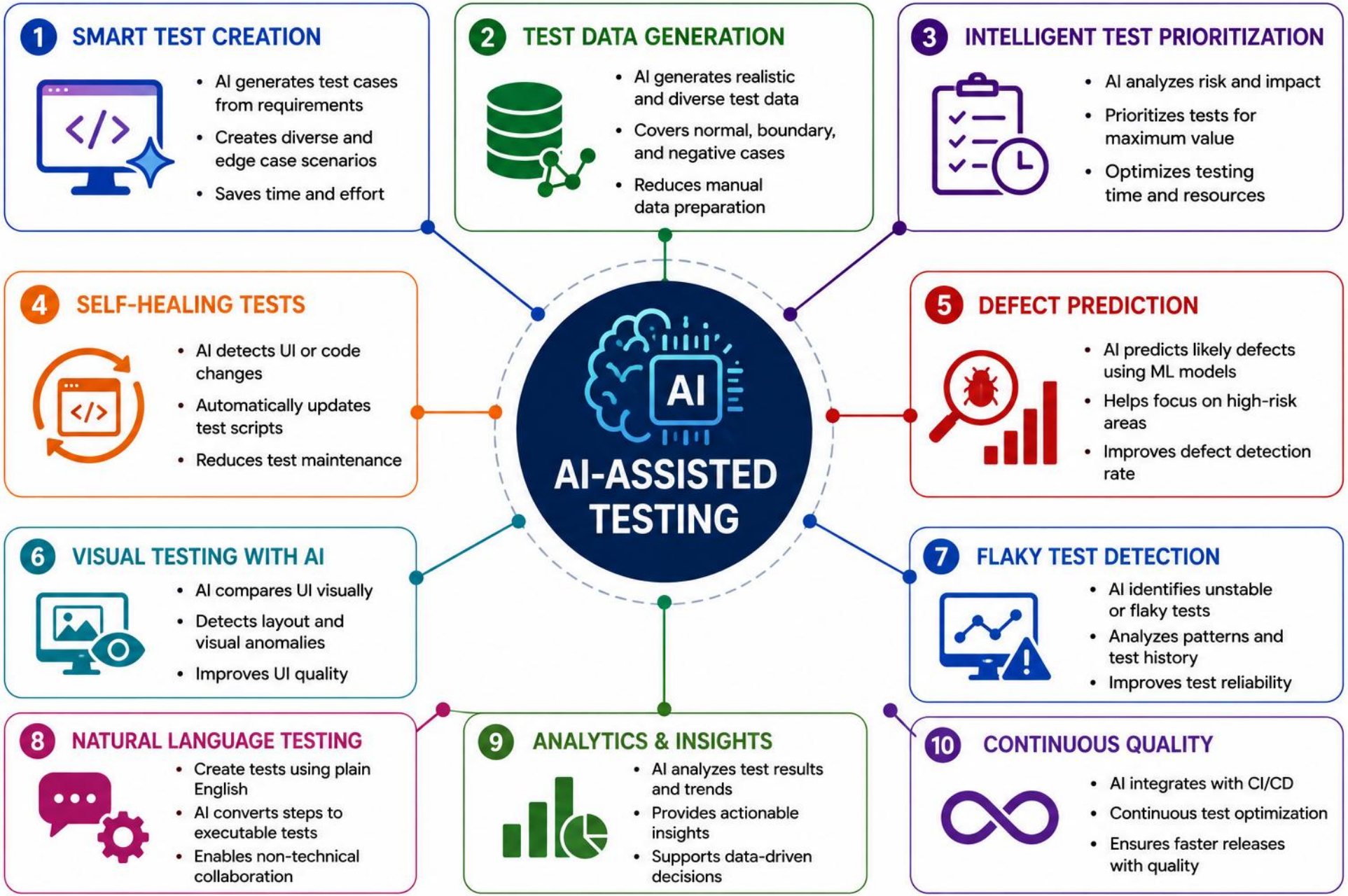
Higher Quality

More Productivity

Lower Costs

Continuous Delivery

KEY TAKEAWAY Testing automation isn't just about writing tests — it's about delivering value faster and building software users trust.



HOW COPILOT SUPPORTS TESTING

Six ways Copilot supports your testing work — from generating the first test to requiring your review.

WHAT COPILOT DOES AT EACH STAGE

Generate Tests

Creates unit tests directly from your code and its behavior.

Suggest Edge Cases

Surfaces boundary, negative, and failure scenarios to cover.

Create Mocks

Builds mocks and stubs so dependencies can be tested in isolation.

Explain Failures

Explains why a test failed and what to check first.

Suggest Refactoring

Proposes cleaner structure without changing behavior.

Require Human Review

Every suggestion needs your review before you trust it.

WHAT YOU WILL LEARN

Leverage AI

Generate tests, cases, and mocks.

Improve Quality

Catch edge cases and gaps early.

Automate Repetitive Work

Focus your time on high-value review.

Enterprise Ready

Reliable, compliant testing at scale.

KEY TAKEAWAY Copilot handles the repetitive parts of testing — you still decide what's correct, secure, and worth shipping.

GENERATING UNIT TESTS

GitHub Copilot helps you quickly generate meaningful unit tests for methods, classes, and services.

HOW TO GENERATE TESTS

- Select the method, class, or file you want to test.
- Use a prompt or comment asking Copilot to generate tests.
- Press Tab to accept, then review the generated assertions.
- Refine mocks, edge cases, and naming as needed.

EXAMPLE TEST (JUNIT 5 + MOCKITO)

```
@Test
void getUserById_existingUser_returnsUser() {
  User expected = new User(1L, "John");
  when(repository.findById(1L))
      .thenReturn(Optional.of(expected));
  User result = service.getUserById(1L);
  assertEquals("John", result.getName());
  verify(repository).findById(1L);
}
```

BEST PRACTICES

Test Behavior

What it does, not how.

Meaningful Names

Describe scenario + result.

Independent Tests

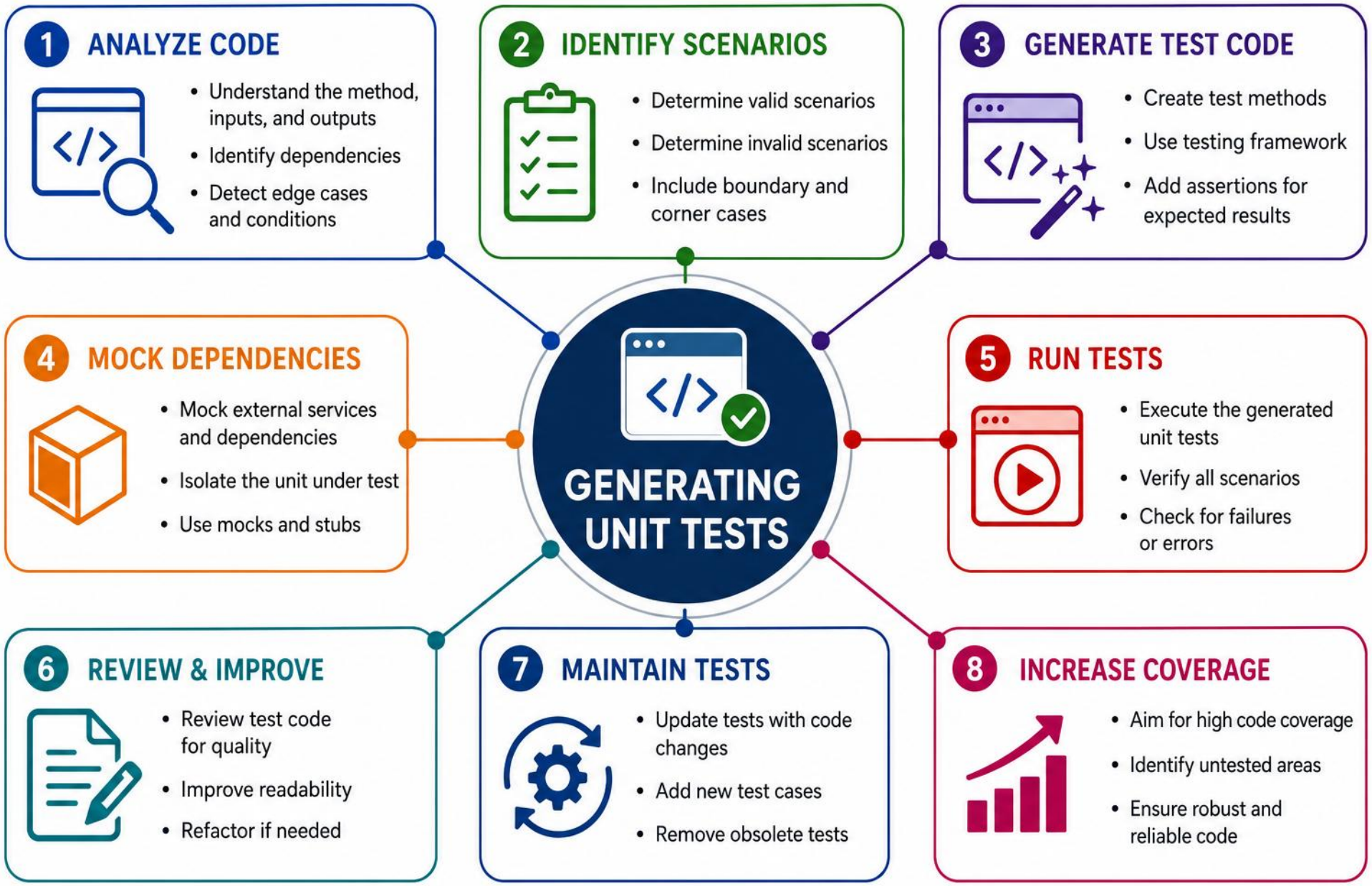
Each test runs in isolation.

Always Review

Validate before trusting it.

Try a specific prompt: "Generate JUnit 5 tests for CustomerService.updateStatus. Use Mockito for CustomerNotifier. Cover valid update, unknown customer, null status, duplicate status update, and notifier failure. Follow Arrange-Act-Assert and use descriptive test names."

KEY TAKEAWAY Copilot accelerates unit test creation, but your understanding ensures the tests are correct and maintainable.



GENERATING EDGE CASES

Edge cases ensure your code is robust and works correctly under unusual or unexpected conditions.

CATEGORY	EDGE CASE	INPUT EXAMPLE	EXPECTED RESULT
`Null`	customerId is null	customerId = null	IllegalArgumentException
`Empty`	customerId is an empty string	customerId = ""	Validation failure
`Missing customer`	Customer ID not found	Unknown ID	CustomerNotFoundException
`Null status`	Status argument is null	status = null	Validation failure
`Same status`	Update repeats the existing status	Existing status repeated	No unnecessary notification
`Notifier failure`	Notifier throws while sending	Mock throws exception	Defined recovery or propagation
`Duplicate customer`	Same ID inserted twice	Same ID inserted twice	Reject duplicate

BEST PRACTICES

Know Your Boundaries

Min, max, null, empty, invalid.

Think Like a User

How might they misuse it?

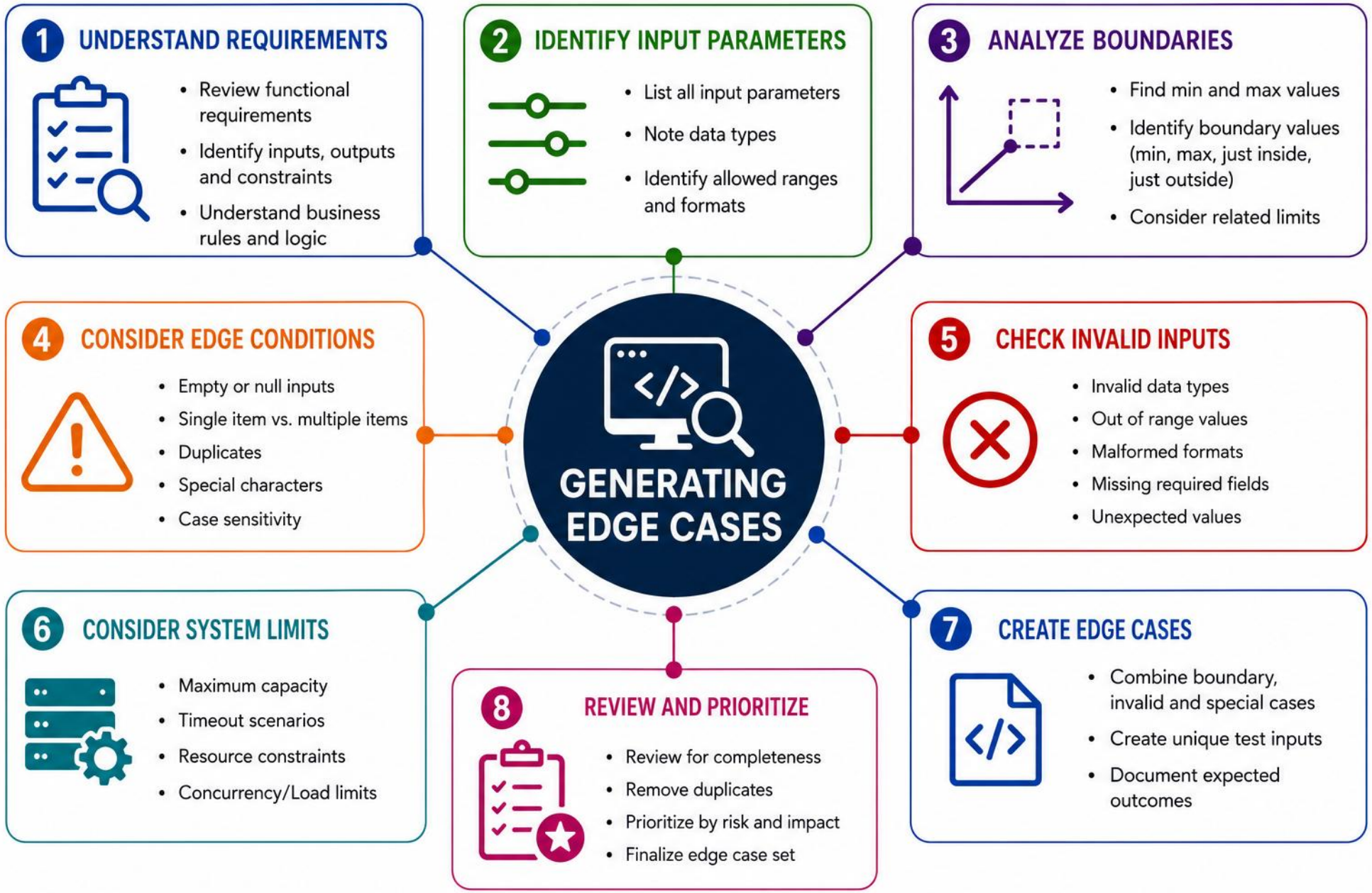
Happy & Sad Paths

Success and failure scenarios.

Review & Refine

Add new edge cases as code evolves.

KEY TAKEAWAY Edge cases turn good code into great code — think broader, test deeper, deliver production-ready software.



GENERATING MOCK OBJECTS

GitHub Copilot helps you create mock objects and test doubles so you can test in isolation with predictable behavior.

WHAT COPILOT GENERATES

- Suggests the right mock objects for your dependencies.
- Generates stubs with default or custom return values.
- Sets up interactions — calls, arguments, order.
- Helps mock exceptions, timeouts, and failures.

EXAMPLE (MOCKITO)

```
@Mock
private PaymentGateway paymentGateway;

when(paymentGateway.charge(order))
  .thenReturn(new PaymentResult(true));
```

BEST PRACTICES

Mock External Deps

Not the class under test.

Mock Behavior

Not internal state.

Stub Only What's Needed

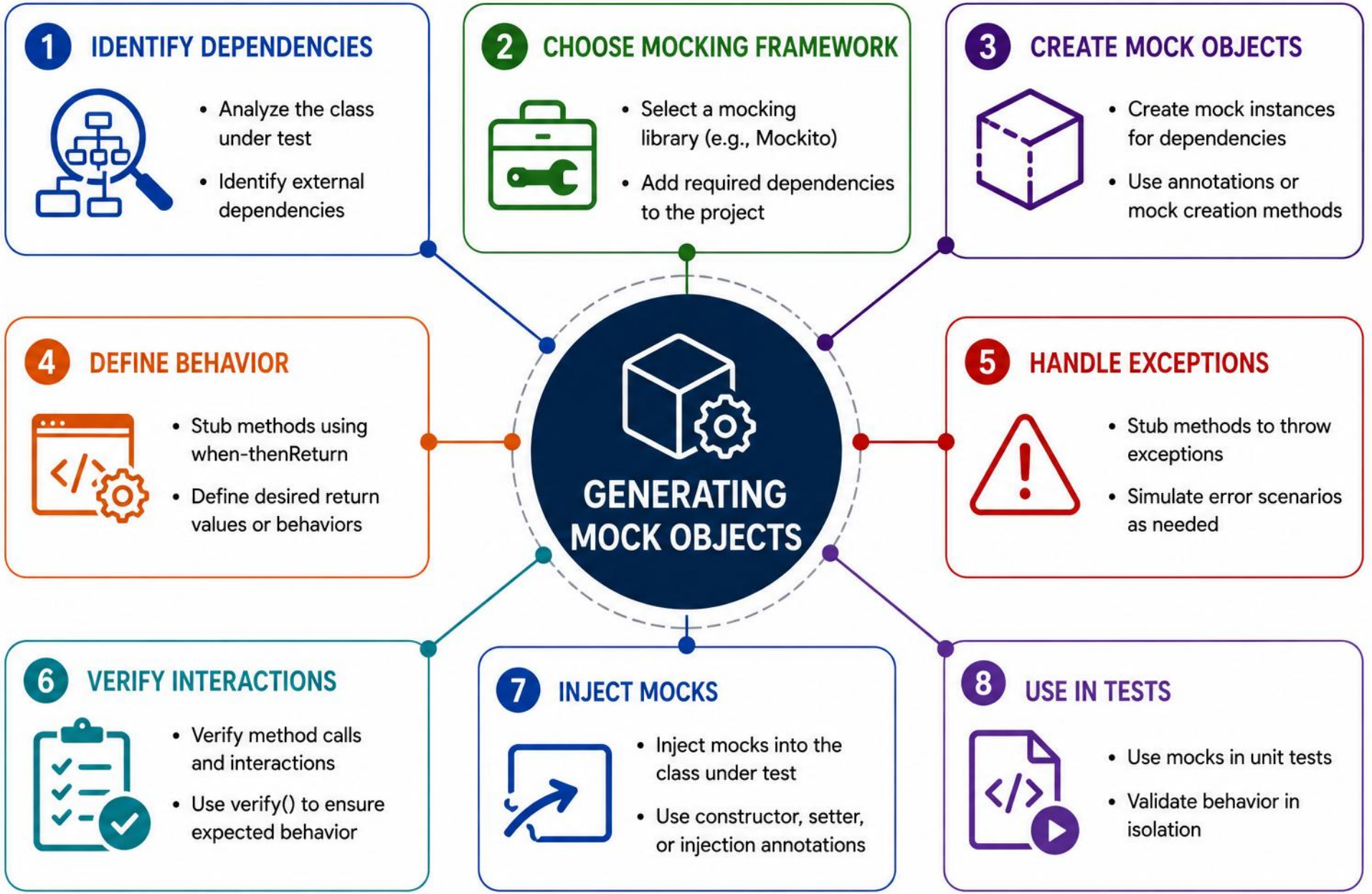
Explicit return values.

Keep Tests Focused

One behavior per test.

Prefer fresh mocks and isolated test setup for each test. Avoid shared mutable state; use reset only in unusual cases.
Verify meaningful interactions only — don't mock value objects or simple domain classes.
Avoid over-specifying implementation details; prefer fakes when they produce clearer tests.

KEY TAKEAWAY Mock objects help you isolate units, control dependencies, and write fast, reliable tests — mock behavior, not implementation.



REFACTORING WITH AI

Copilot proposes refactorings, tests, and documentation — helping you improve code without changing behavior.

HOW COPILOT HELPS

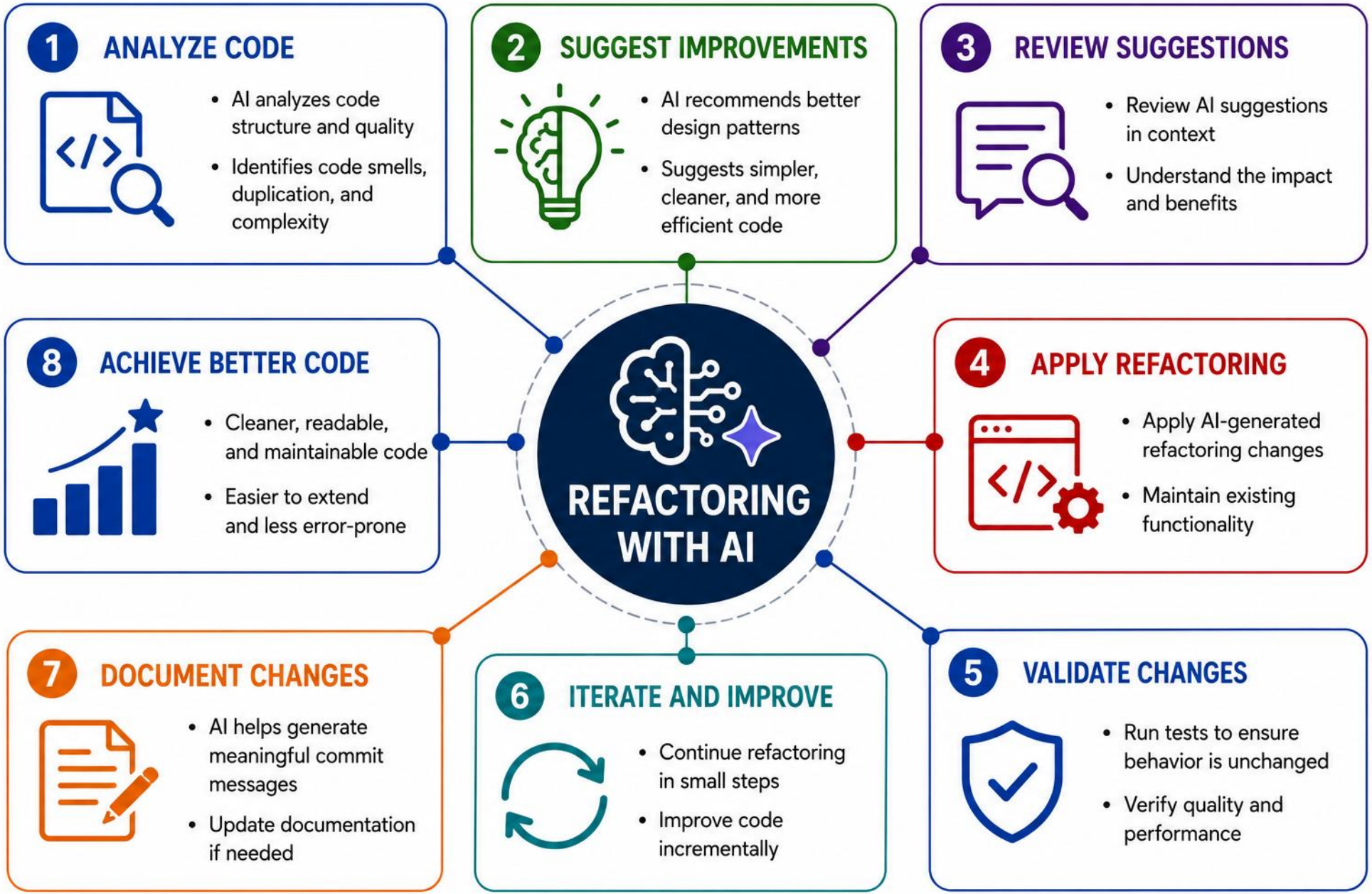
EXAMPLE PROMPTS

- *"Refactor this method for readability without changing observable behavior. Preserve public signatures and existing exception behavior."*
- *"Analyze this method for performance bottlenecks. Do not change the code yet. Explain complexity, allocation hotspots, and what measurements would justify optimization."*
- *"Add null and empty checks with meaningful exceptions."*
- *"Suggest unit tests for this method using JUnit 5."*

ALSO REMEMBER

Refactor incrementally, write tests first, follow SOLID/DRY design principles, use version control, and measure impact (complexity, coverage, performance).

KEY TAKEAWAY Refactor in small, tested steps — confirm behavior is unchanged before and after every change.



DETECTING CODE SMELLS

GitHub Copilot helps you identify code smells early so you can reduce technical debt and improve maintainability. Treat size thresholds as prompts for review, not automatic proof of a smell — consider complexity, cohesion, domain clarity, and change frequency too.

CODE SMELL	WHAT IT MEANS	ILLUSTRATIVE EXAMPLE	COPILOT SUGGESTION
`Long Method`	Method does too many things	Method > 20-30 lines	Extract method, split responsibilities
`Large Class`	Too many fields/methods	Class > 15-20 methods	Extract class, apply SRP
`Duplicated Code`	Same code repeated	Similar blocks in many places	Extract method, reuse components
`Long Parameter List`	Too many parameters	> 4-5 parameters	Introduce Parameter Object
`Magic Number/String`	Hardcoded values	e.g. 0.90, "VIP"	Use constants, enums

HOW TO RESPOND

Detect Early

Review as you write code.

Understand the Smell

Read the explanation.

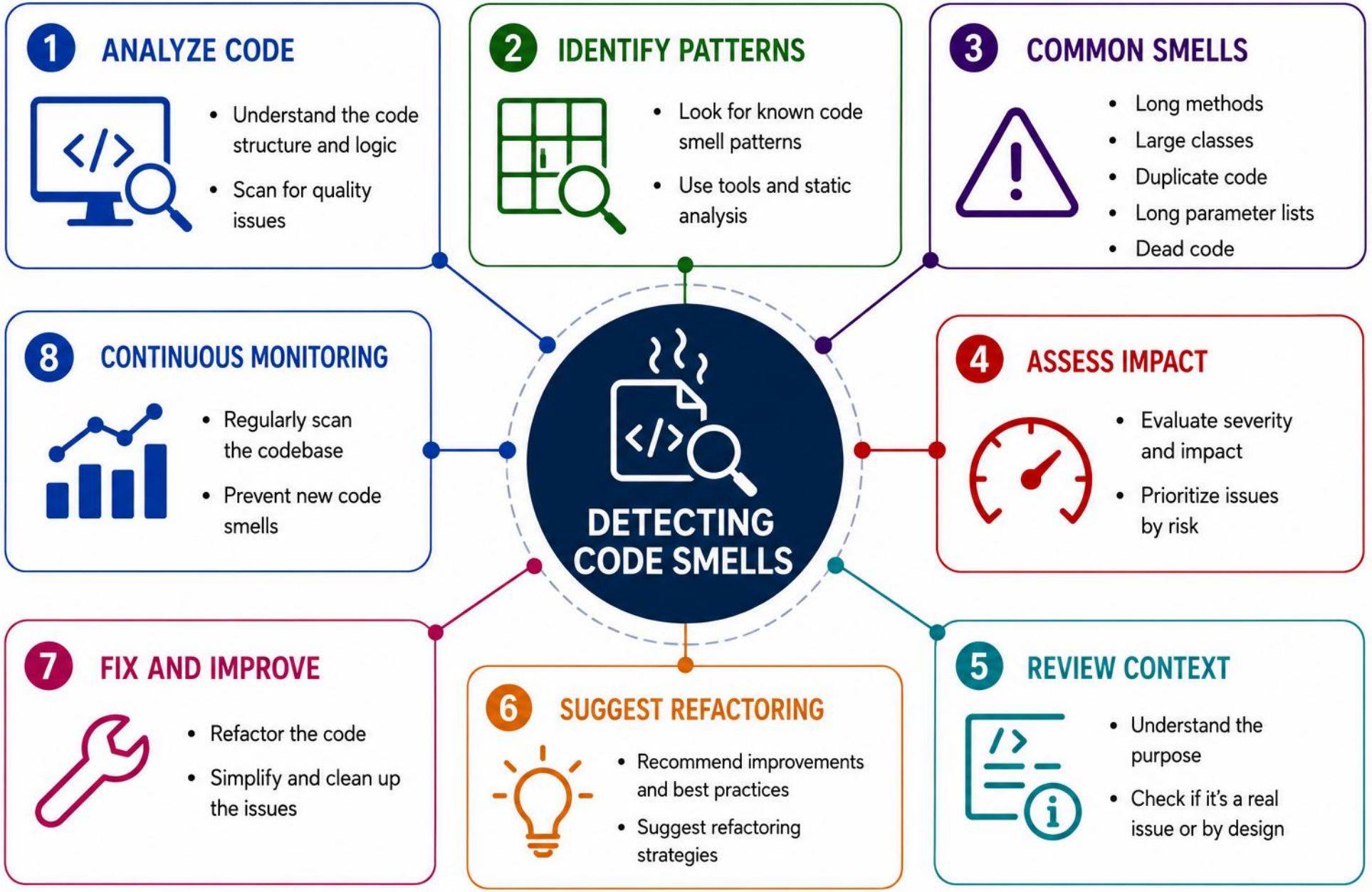
Refactor Incrementally

Small, safe changes.

Back with Tests

Ensure behavior stays the same.

KEY TAKEAWAY Code smells are warnings, not errors. Detect them early, refactor with confidence, and keep your codebase future-ready.



REFACTORING QUALITY CHECKLIST

A shared checklist for evaluating any Copilot-assisted refactor — from naming to behavior preservation.

HOW COPILOT HELPS

- Suggests clear, meaningful names for variables, functions, classes.
- Simplifies complex expressions and conditions into simpler steps.
- Extracts and organizes code into focused methods.
- Adds helpful comments that explain intent, not mechanics.

BEFORE / AFTER

```
// Before
double c(double p,double r,int y,int m){...}

// After
double computeCompoundInterest(
  double principal, double ratePercent) {...}
```

QUALITY CHECKLIST: SIX THINGS TO VERIFY

Naming & Clarity

Names reveal intent without extra comments.

Method/Class Size

Size is a prompt to look closer, not an automatic rule.

Duplication

Extract shared logic instead of repeating it.

Coupling & Abstractions

Depend on interfaces, not another class's internals.

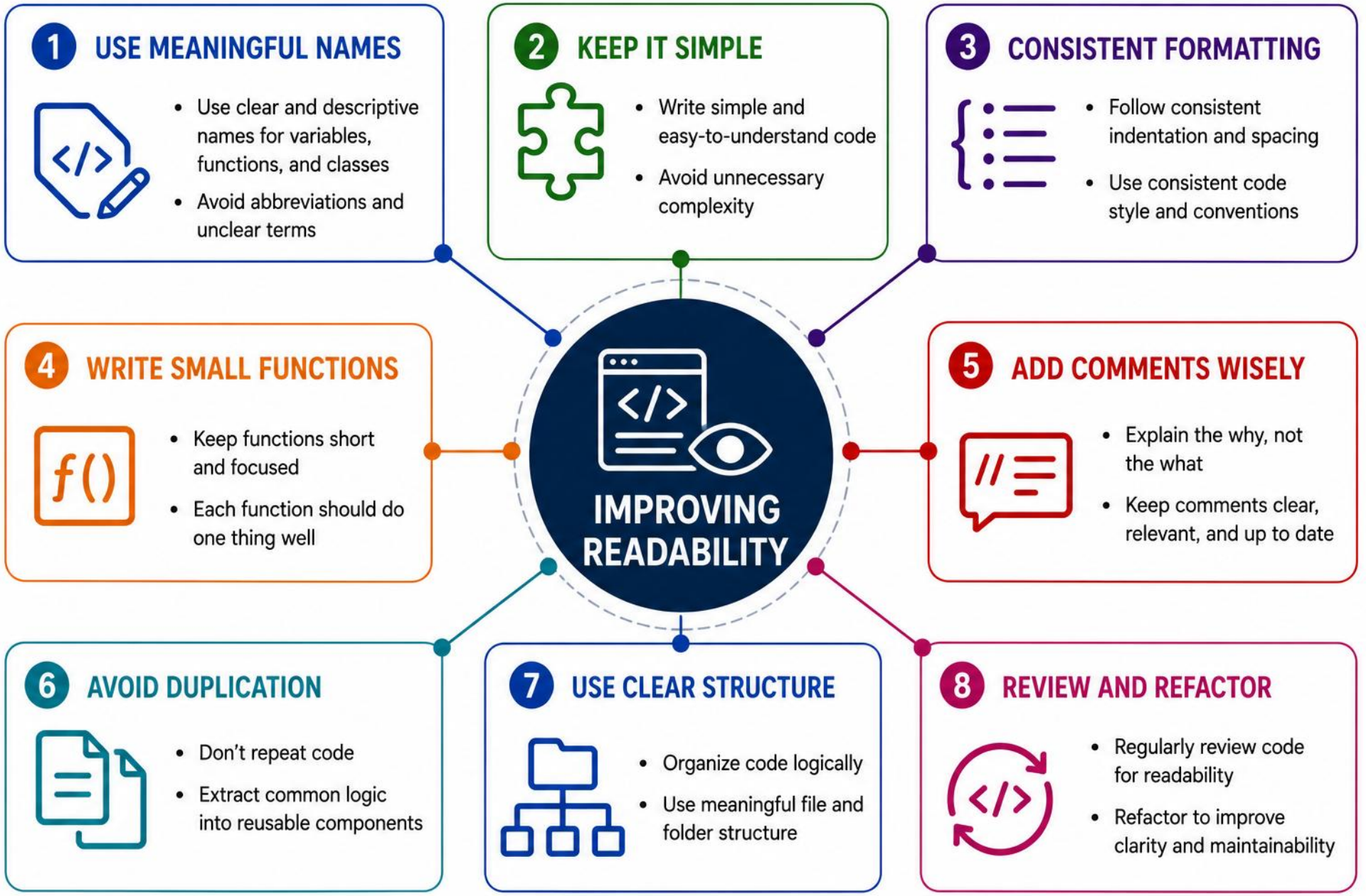
Domain Types

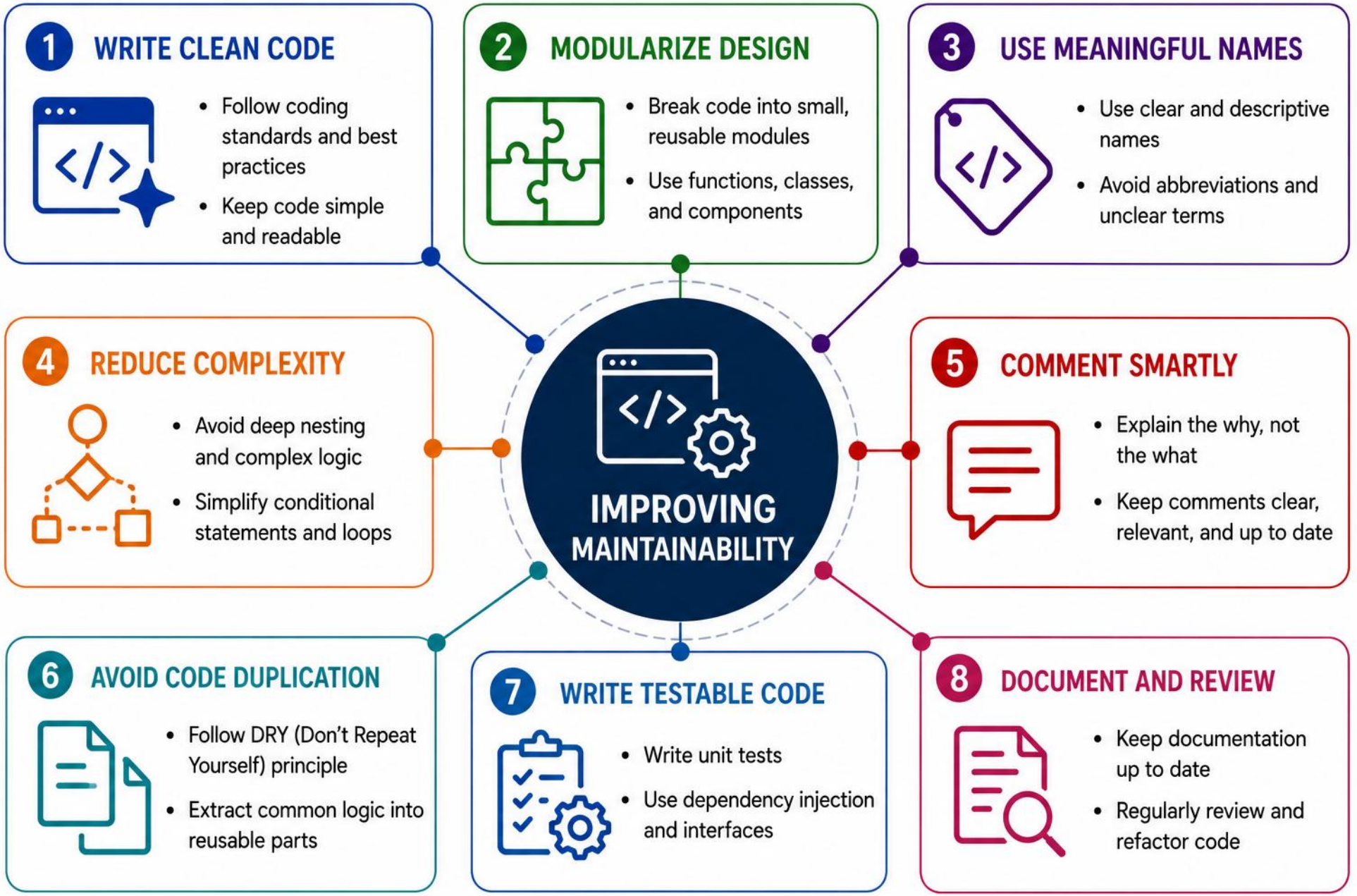
Use value objects and enums, not raw primitives.

Tests & Behavior Preservation

Existing tests still pass; behavior doesn't change.

KEY TAKEAWAY Readable, well-structured code is a long-term investment — checked against a clear standard, not guesswork.





REVIEWING AI-GENERATED TESTS AND REFACTORS

AI can generate great tests and refactors — but you are the final reviewer. Different questions apply to each.

WHY YOU MUST REVIEW

Correctness

Works as intended.

Security

No vulnerabilities.

Quality

Consistent with standards.

Maintainability

Easy to read and test.

Confidence

Improves your team.

FOR AI-GENERATED TESTS

- Fails when the production code is broken?
- Assertions are meaningful, not just non-null checks?
- Tests are independent of each other?
- Mocks aren't hiding real behavior?
- Negative and boundary cases are covered?
- Verifies public behavior, not internals?

FOR AI-ASSISTED REFACTORS

- Observable behavior is preserved?
- All existing tests still pass?
- Complexity has actually improved?
- Public APIs are unchanged?
- Exceptions, logging, and transactions are unchanged?
- The diff is smaller and easier to understand?

GOLDEN RULE AI writes the first draft. You ensure it's production-ready.

KEY TAKEAWAY AI is your assistant, not a replacement — review its tests and refactors as carefully as you would your own.

WHERE AI-ASSISTED UNIT TESTING FITS

A repeatable flow from requirements to release, with quality gates and the metrics that matter. Toolchain examples: Docker/Kubernetes, Jira/Azure DevOps, Selenium/Postman/JUnit, JMeter/k6/OWASP ZAP, Datadog/Grafana/Allure — versions per the course setup guide.

Derive test conditions from acceptance criteria.

Fast, isolated, run on every commit.

Verify components work together correctly.

Automated checks block risky changes.

Ship with confidence and monitor in production.

CORE QUALITY METRICS

Test Pass Rate

Share of tests passing on the latest build.

Coverage (as a Signal)

Shows what ran, not what was verified — track the trend.

Mutation Score (if applicable)

How many injected defects the tests actually catch.

Flaky-Test Rate

Tests that pass or fail without any code change.

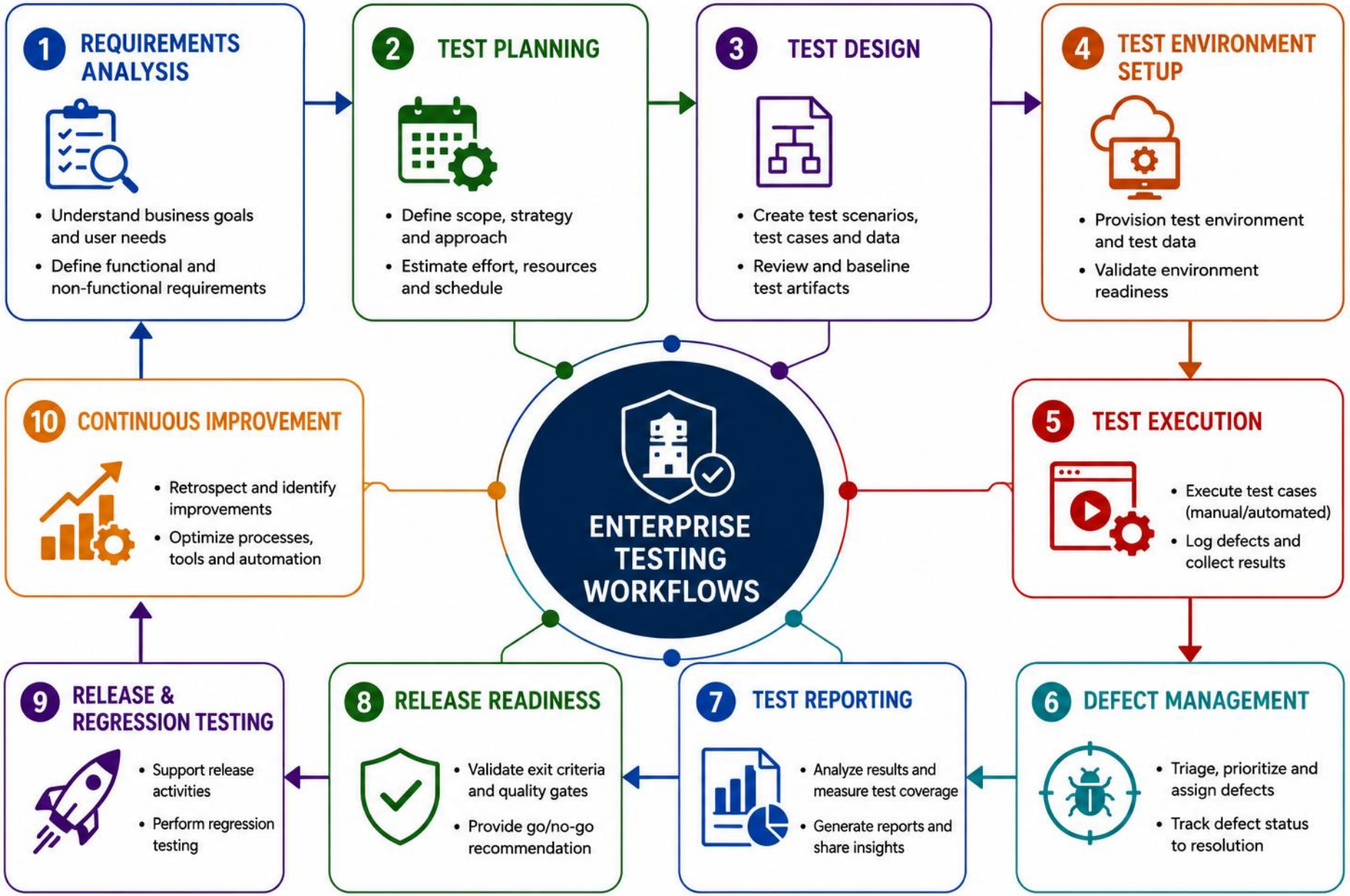
Defect Escape Rate

Bugs found in production that tests should have caught.

Test Execution Time

How long the suite takes — keep feedback loops fast.

KEY TAKEAWAY Track pass rate, coverage trend, and flaky/escape rates together — no single metric proves quality on its own.



LAB OVERVIEW – AI-ASSISTED TEST GENERATION

You extend the Northstar CRM's CustomerService with a safety-net test suite and a cleaner design.

SAMPLE CODE UNDER TEST

```
public class CustomerService {
  private final List<Customer> customers;
  private final CustomerNotifier notifier;
  public Customer updateStatus(
      String id, CustomerStatus s) {...}
}
```

LAB WORKFLOW

- Copy Lab 10's CustomerService; add JUnit 5 + Mockito.
- Generate tests with Copilot; reject weak assertions.
- Extract CustomerNotifier; verify it with a mock.
- Review coverage gaps; write acceptance guidelines.

LAB OBJECTIVES

Generate Unit Tests

For Customer and CustomerService.

Reject Weak Tests

Spot assertions that can't fail.

Extract & Mock

CustomerNotifier, verified with Mockito.

Review & Refine

Coverage gaps, acceptance guidelines.

Tools: GitHub Copilot, IntelliJ IDEA or VS Code, JDK 21+, Maven, JUnit 5, Mockito — use the versions supplied by the starter POM or the current course setup guide.

KEY TAKEAWAY **Estimated time: 45 minutes. Build the safety net first, then refactor with confidence.**

LAB TASKS AND DELIVERABLES

Apply AI to generate, enhance, and validate tests for enterprise-quality code.

#	TASK	DETAILS	FORMAT
1	Entity & Service Tests	CustomerTest and CustomerServiceTest via Copilot.	.java
2	Reject False Confidence	Catch and replace an assertion that can't fail.	.java
3	Extract & Mock Notifier	CustomerNotifier interface, verified with Mockito.	.java
4	Refactor Duplication	Extract validateCustomerId(); confirm with tests.	.java
5	Coverage Gap Review	List covered/untested methods; note honest gaps.	.md
6	Acceptance Guidelines	5-point checklist for AI-generated tests/refactors.	.md
7	Mutation Check (Optional)	Break a production result, confirm the test fails, then restore it and record what the test caught.	.md

ACCEPTANCE CRITERIA

- All ~7 tests pass with `mvn clean test`; CustomerNotifier is extracted and verified via Mockito; at least one false-confidence test is rejected and documented; coverage gaps are documented honestly, not chased to 100% — coverage shows which code executed, not whether behavior was verified correctly.

KEY TAKEAWAY **Ship code you understand and can defend — tests, mocks, and refactors included.**

AI-ASSISTED TESTING: DEFINITION OF DONE

Before you call a test suite or refactor finished, check it against this list.

1. Tests verify public behavior, not internal implementation.
2. Assertions can actually fail — no tautologies.
3. Happy, negative, and boundary paths are covered.
4. Mocks isolate only true external dependencies.
5. Tests are deterministic and independent of each other.
6. Refactoring preserves observable behavior.
7. Existing and new tests all pass.
8. Coverage gaps are documented honestly, not hidden.
9. AI suggestions are understood and reviewed before accepting.
10. Final code meets team standards and conventions.

KEY TAKEAWAY A test suite is done when it verifies behavior, fails honestly, and you understand every AI-assisted line in it.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of AI-assisted testing and refactoring concepts.

1. What's the primary benefit of AI for test generation?

A. It replaces human testers completely
B. It increases test coverage and speed
C. It eliminates the need for unit tests
D. It removes the need for code reviews

2. Which tool creates mocks for external dependencies?

A. JUnit
B. Mockito
C. JaCoCo
D. SonarQube

3. What's the key advantage of testing in a CI pipeline?

A. Manual test execution
B. Early feedback and continuous validation
C. Reduced need for edge cases
D. Elimination of unit tests

4. Which AI-generated test creates false confidence?

A. A test with a meaningful expected exception
B. A test that only asserts `result != null`
C. A test verifying repository interaction
D. A parameterized boundary test

KEY TAKEAWAY AI accelerates test generation — coverage and speed improve, but human review still ensures quality.

KNOWLEDGE CHECK (2 OF 2)

Four more questions, plus the full answer key.

5. What's the goal of the AAA pattern in unit tests?

A. Arrange, Act, Assert
B. Analyze, Act, Adjust
C. Automate, Analyze, Assess
D. Arrange, Automate, Assert

6. Which is a best practice for AI-assisted testing?

A. Use AI output without any changes
B. Keep tests independent and atomic
C. Test only the happy path
D. Avoid writing unit tests manually

7. Why are edge cases important in testing?

A. They test the happy path
B. They help uncover hidden defects
C. They validate security policies
D. They reduce the number of test cases

8. What should you do with AI-generated tests?

A. Run them without any changes
B. Accept them if they compile
C. Review, refine, and validate them
D. Use them only in production

ANSWER KEY

- **1-B 2-B 3-B 4-B 5-A 6-B 7-B 8-C**

KEY TAKEAWAY AI accelerates testing — higher quality, more productivity, greater confidence — but review, refine, validate is still yours.

"AI is your assistant, not a replacement."

MODULE 11 COMPLETE

GitHub Copilot for Testing and Refactoring

You can now use AI to generate tests, uncover edge cases, and refactor code — while staying the final, critical reviewer.